“So... What Even Is Agentic AI?”

Agentic AI for Dummies

Thiliban Manivarma

Agentic AI

- An AI system that can autonomously plan, decide, and act to achieve a goal.
- It uses a Large Language Model (LLM) to reason and a set of tools (functions, APIs) to execute actions.
- It runs in a loop: it thinks about the next step, executes it, observes the result, and repeats until the task is complete.

Why Are We Doing This?

- Because AI isn't just for big companies.
- I wanted to show that you can build your own intelligent assistant one that runs entirely on your laptop, never phones home, and actually *does* useful work (not just chat).
- Think of it as teaching a computer to be a helpful lab partner who:
  - Understands your questions in plain English
  - Knows when to grab a tool (your Python code)
  - Gives you answers you can actually use

What Can You Do With It?

Ask questions like:

- *"Is the sequence ALAGLY hydrophobic?"*
- *"Compare the hydrophobicity of ILVFW and EDNQK."*
- *"Explain what hydrophobicity means and give an example."*

- The AI will figure out what you need, run the right Python function behind the scenes, and reply with a clean answer just like ChatGPT, but 100% local and free.

Main components of vanilla/basic agentic AI:

Component	What It Is (Kid Version)	Simple Analogy
1) Large Language Model (LLM) (Brain)	A super-smart friend who has read almost every book in the world and can chat about anything. They're great at understanding what you mean but have no hands to actually do physical tasks.	Think of a chef who only reads recipes aloud. The chef knows what ingredients are needed and the steps, but cannot chop, stir, or bake anything themselves.

- The LLM's job is **to understand your plain English** and **figure out which tool to use**, with **what inputs**.
- It doesn't run tools itself, it simply hands a request to the **agent code**, which then calls the real Python function (or MCP server) for you.
- So the LLM is the brain that reads your mind, and the agent is the hands that do the actual work.

Component	What It Is (Kid Version)	Simple Analogy
2) Tool (Hands)	A small machine or helper program that is excellent at one specific job like a toaster that only makes toast, or a mixer that only blends. It doesn’t understand words; it just does its one task perfectly when asked.	Imagine a kitchen appliance like a blender. The blender doesn't know why you want a smoothie; it just blends whatever you put in when you press the button.

- Tools do the real work. They run your Python code to calculate, fetch files, search databases, or call other software that the LLM can't execute by itself. Basically, **your python script will act as tool.**

- The LLM decides what needs to be done, and the tool actually performs that action, then hands the result back so the AI can answer you in plain English.

Component	What It Is (Kid Version)	Simple Analogy
3) Agentic AI (The Whole System)	The complete kitchen team: the chef (LLM) reads the recipe, decides to use the blender (tool), puts in the fruit (your request), turns it on, and serves you the smoothie. The team works together from start to finish.	Picture a smart kitchen robot with a cookbook reader, a blender, and an oven. You say “make a smoothie,” and the robot figures out the steps, picks the right appliance, uses it, and hands you the drink.

- Agentic AI is the complete system that wires the LLM (the brain) to your tools (the hands) through a decision-loop.
- **The AI can autonomously plan which tool to use, call it, look at the result, then decide whether to call another tool or give you a final answer.** All without you telling it every step.
- In short: it turns a chatbot into a digital lab assistant that thinks, acts, and learns in a continuous cycle.

What Will You Learn?

Main components of vanilla/basic agentic AI:

1. Local LLMs: How to run an AI brain on your own computer using Ollama (open-source)
2. Function Calling: How to teach an AI to use your Python code as a tool
3. AI Agents: How to build a loop where the AI thinks → acts → observes → answers

Outcome: By the end, you won't just use AI — you'll understand how to build one that works for you

Freedom: No cloud. No API keys. No waiting. Just you, your code, and an AI that follows your instructions

Why Should Life Sciences & Biotech Beginners Care?

- **Automate routine analysis** – Instantly calculate hydrophobicity, molecular weight, pI, or binding scores just by asking a question in plain English.
- **Connect to biological databases** – Use MCP tools to fetch protein structures (PDB), sequences (UniProt), or compounds (PubChem) without writing complex API code.
- **Keep data private & local** – All processing stays on your laptop; sensitive research data never leaves your machine.
- **Reuse your existing Python scripts** – Teach the AI to call your own analysis tools (docking, QSAR, sequence alignment) without rewriting anything.
- **Build AI that *does*, not just *chats*** – Create agents that plan multi-step tasks (fetch target → compute properties → compare results) autonomously.
- **Future-proof your skills** – Agentic AI is driving modern drug discovery; understanding these foundations puts you ahead in computational biology and cheminformatics.

Main workflow of vanilla/basic agentic AI

User Input (YOU*)

LLM (Brain) Understands the question & decides if a tool is needed

If Tool needed → Tool Call Request (LLM asks for a specific tool + inputs)

Agent Loop (Orchestrator) Intercepts the request & runs the real tool

Tool Execution (Python function / MCP server) Performs the actual calculation or data fetch

Tool Result (Output sent back to LLM), LLM (again) → Reads the result and forms a final answer

If No tool needed → Final Answer → User sees reply

Final Answer → User sees reply

You ask (USER) → Waiter takes note (Agent) → Chef reads ticket & chooses appliances (LLM) → Appliances do the work (Tools) → Chef assembles dish (LLM) → Waiter serves it (Agent) → You get your pizza ☺ (USER)

Conclusion

What You Now Understand

1. **Local LLMs** give you a private, free AI brain running on your own machine via Ollama.

2. **Tools** are simply your Python functions. The LLM never sees your code, only a description card that tells it what each tool does.

3. **Agentic AI** connects the brain (LLM) to the hands (tools) through a simple loop: **think → decide → act → observe → answer**.

“One simple idea wrapping a Python script with an LLM unlocks a new way to build AI that doesn't just chat, but *does*”

You now hold the blueprint. Build something useful, share it, and keep experimenting ☺

“Protein Hydrophobicity AI Agent”

Thiliban Manivarma

Step-by-Step: How the Protein Hydrophobicity Agent Works

Step 1 – The Tool (Python Function)

```
def calculate_hydrophobicity(sequence: str) -> dict:
    ...
```

- This is the tool. A normal Python function that knows the science.
- It takes an amino acid sequence (like "ALAGLY"), looks up each letter in the Kyte-Doolittle scale, and returns the average score + classification.
- The LLM never sees this code. It only knows the tool exists and what it needs.

Step 2 – The Menu Card (Tool Definition)

```
tools = [{
    'type': 'function',
    'function': {
        'name': 'calculate_hydrophobicity',
        'description': 'Calculate protein hydrophobicity ...',
        'parameters': {
            'properties': {
                'sequence': {'type': 'string', ...}
            },
            'required': ['sequence']
        }
    }
}]
```

This JSON block is like a restaurant menu. It tells the LLM:

- *What* the tool is called.
- *What it does* (description).
- *What information it needs* (a sequence as a string).

The LLM reads this menu to decide when and how to use the tool.

Step 3 – The Smart Loop (Agent Brain)

```
def run_agent_clean(user_query: str, ...):
    messages = [{'role': 'user', 'content': user_query}]
    while True:
        response = ollama.chat(model=model, messages=messages, tools=tools)
        ...
```

- Send the user's question to the LLM (brain).

The LLM either:

  - Answers directly → done.
  - Asks to run a tool → we run it and send the result back.

- If a tool was used, the LLM reads the result and decides whether to answer or ask for another tool.
- Repeat until the LLM gives a final answer.

Step 4 – The Friendly Interface (Chat)

```
def chat():
    print("... PROTEIN HYDROPHOBICITY AI ASSISTANT ...")
    while True:
        query = input("You: ")
        answer = run_agent_clean(query)
        print("AI:", answer)
```

- A simple text chat that lets you type questions.
- Calls the agent loop for each query and prints the answer.t
- Type quit to exit.

Summary – One Complete Cycle

- **You ask** → *“What is the hydrophobicity of ALAGLY?”*
- **LLM thinks** → *“I need the* calculate_hydrophobicity *tool with sequence* ALAGLY.”
- **Agent runs** → calls calculate_hydrophobicity("ALAGLY") → gets 1.15 (hydrophobic)
- **LLM answers** → *“The average hydrophobicity of ALAGLY is 1.15, which makes it hydrophobic.”*

The magic? ☺ The brain (LLM) decides, the hand (Python function) does the work, and the loop connects them automatically.